LAPORAN ETS ROBOTIKA - A081

Oleh:

Shahzada Sabih Anandian Hartanto	23081010209
Muhammad Abdul Aziz	23081010229
Mikail Zamakhsyari	23081010115
Mochamad David Ainur Rochman	23081010239
Iqbal Zahy Adam	23081010168

ROBOTIKA A-081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

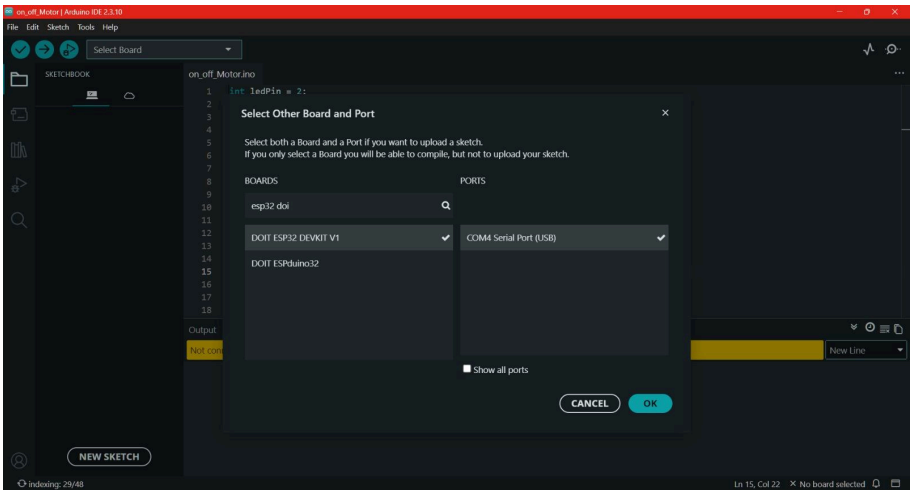
1. Basic Robotika - Kit iMcLab.....3
1.1. Tata Cara.....3
1.1.1. Setup Board Manager.....3
1.1.3.1. Code.....3
1.1.3.2. Check Serial Monitor untuk Command.....5
1.2. Hasil.....6
2. Python - Kit iMcLab.....6
2.1. Tata Cara.....6
2.1.1. Setup Board Manager.....6
2.1.2. Sambungkan Device ke Laptop.....7
2.1.3. Start up & Compile.....7
2.1.3.1. Code.....7
2.1.3.2. Check Serial Monitor untuk Command.....8
2.2. Hasil.....9
3. Blynk - Kit iMcLab.....9
3.1. Tata Cara.....9
3.1.1. Setup Board Manager.....9
3.1.2. Setup Library Manager.....10
3.1.3. Menyambungkan Device ke Laptop.....10
3.1.4. Start up & Compile.....11
3.2. Dashboard Blynk Desktop.....19
3.3. Hasil.....20
4. Blynk O.T.A - Kit iMcLab.....20
4.1. Tata Cara.....20
4.1.1. Setup Board Manager.....20
4.1.2. Setup Library Manager.....21
4.1.3. Menyambungkan Device ke Laptop.....21
4.1.4. Code.....21
4.2. Dashboard Blynk OTA.....34
4.3. Update via Blynk Air OTA.....34
4.4. Hasil setelah Update via Blynk Air OTA.....35

1. Basic Robotika - Kit iMcLab

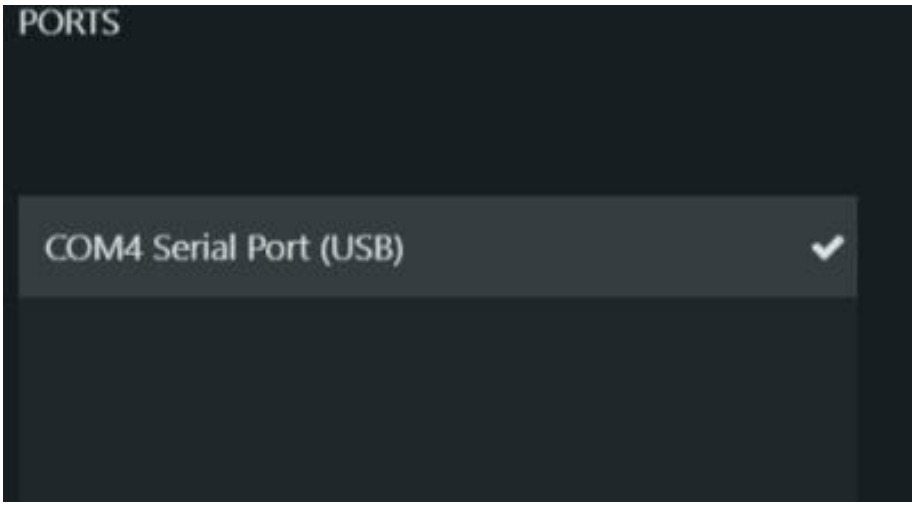
Modul ini memperkenalkan pemrograman dasar robotika pada kit iMCLab (ESP32 dengan driver motor DC, LED indikator, dan sensor RPM)

1.1. Tata Cara

1.1.1. Setup Board Manager



1.1.2. Menyambungkan Device ke Laptop



1.1.3. Start up & Compile

1.1.3.1. Code

```
#include <WiFi.h>
#include <ESPmDNS.h>
#include <WiFiUdp.h>
#include <ArduinoOTA.h>

const char* ssid = "*****";
const char* password = "*****";
```

```
void setup() {
  Serial.begin(115200);
  Serial.println("Booting");
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.waitForConnectResult() !=
WL_CONNECTED) {
    Serial.println("Connection Failed!
Rebooting...");
    delay(5000);
    ESP.restart();
  }

  // Port defaults to 3232
  // ArduinoOTA.setPort(3232);

  // Hostname defaults to esp3232-[MAC]
  ArduinoOTA.setHostname("TheHolyTrinity");

  // No authentication by default
  ArduinoOTA.setPassword("JesusIsLord");

  // Password can be set with it's md5 value as
well
  // MD5(admin) = 21232f297a57a5a743894a0e4a801fc3
  //
ArduinoOTA.setPasswordHash("21232f297a57a5a743894a0
e4a801fc3");

  ArduinoOTA
    .onStart([]() {
      String type;
      if (ArduinoOTA.getCommand() == U_FLASH)
        type = "sketch";
      else  // U_SPIFFS
        type = "filesystem";

      // NOTE: if updating SPIFFS this would be the
place to unmount SPIFFS using SPIFFS.end()
      Serial.println("Start updating " + type);
    })
    .onEnd([]() {
      Serial.println("\nEnd");
    })
    .onProgress([](unsigned int progress, unsigned
int total) {
      Serial.printf("Progress: %u%%\r", (progress /
(total / 100)));
    })
    .onError([](ota_error_t error) {
      Serial.printf("Error[%u]: ", error);
      if (error == OTA_AUTH_ERROR)
Serial.println("Auth Failed");
```

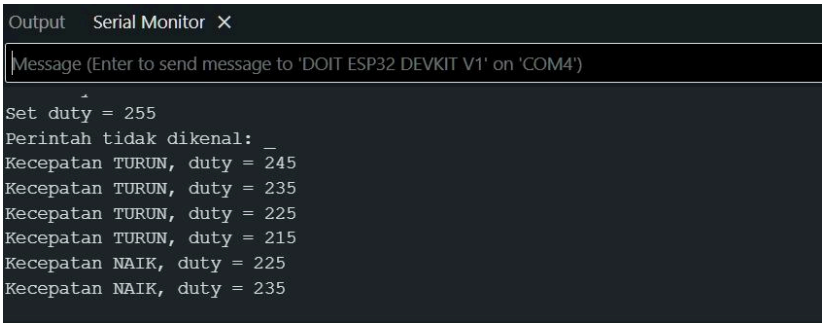
```
      else if (error == OTA_BEGIN_ERROR)
Serial.println("Begin Failed");
      else if (error == OTA_CONNECT_ERROR)
Serial.println("Connect Failed");
      else if (error == OTA_RECEIVE_ERROR)
Serial.println("Receive Failed");
      else if (error == OTA_END_ERROR)
Serial.println("End Failed");
    });

  ArduinoOTA.begin();

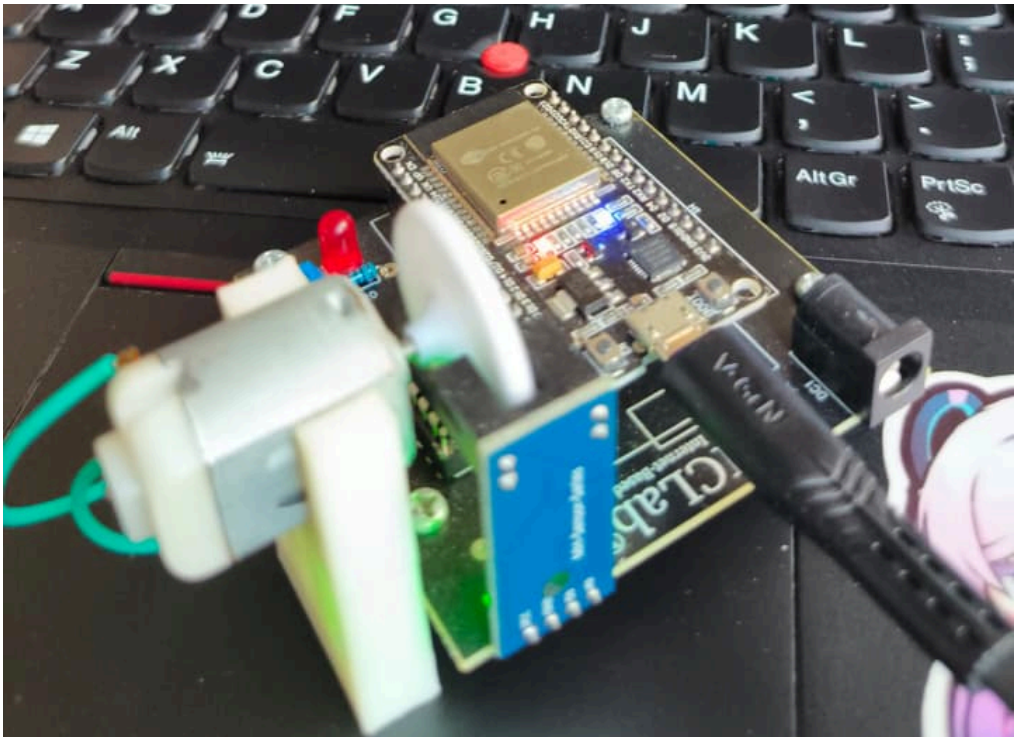
  Serial.println("Ready");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
  pinMode(2, OUTPUT);
}

void loop() {
  ArduinoOTA.handle();
  digitalWrite(2, HIGH);
  delay(250);
  digitalWrite(2, LOW);
  delay(250);
}
```

1.1.3.2. Check Serial Monitor untuk Command



1.2. Hasil

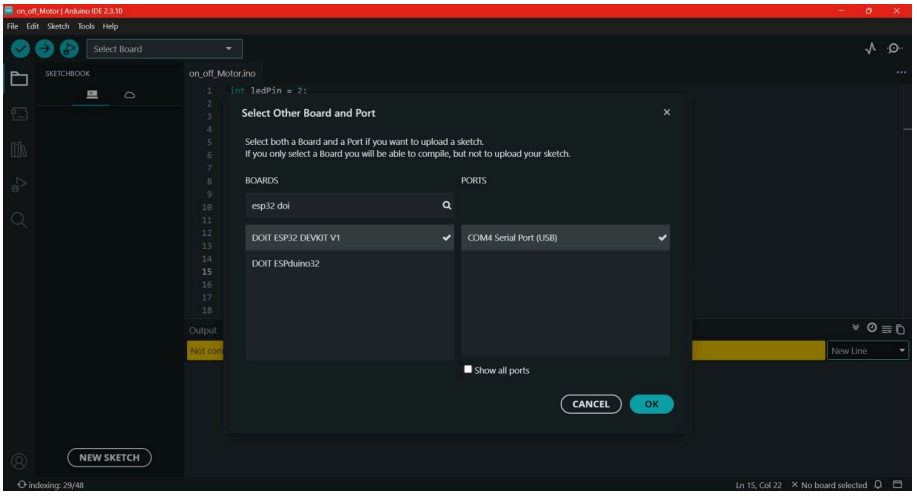


2. Python - Kit iMcLab

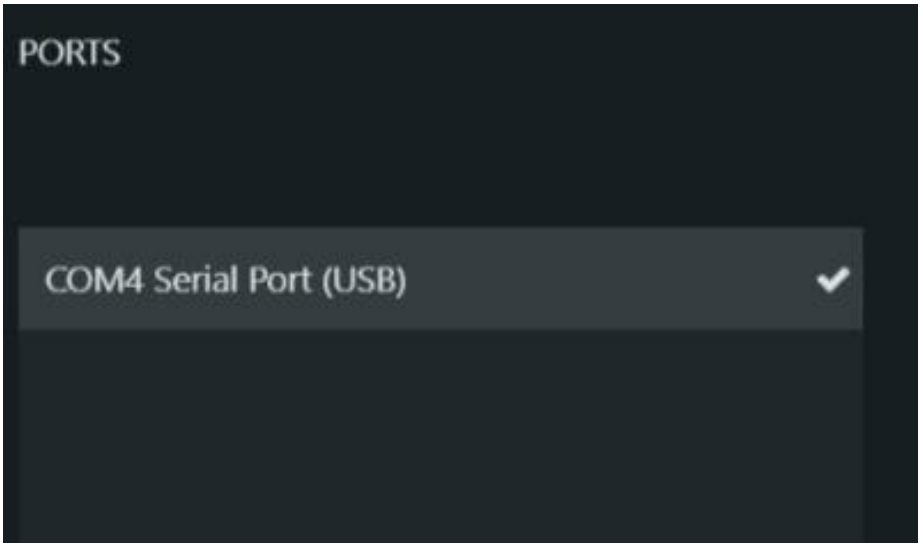
Experiment Modul Python menggunakan Kit iMcLab

2.1. Tata Cara

2.1.1. Setup Board Manager



2.1.2. Sambungkan Device ke Laptop



2.1.3. Start up & Compile

2.1.3.1. Code

```
import imclab
import time
# Connect to Arduino
a = imclab.iMCLab()
print('LED On')
a.LED(100)
# Pause for 1 second
time.sleep(1.0)
print('LED Off')
a.LED(0)
a.close()""
import imclab
import time

# Connect to Arduino
a = imclab.iMCLab()

# Get Version
print(a.version)

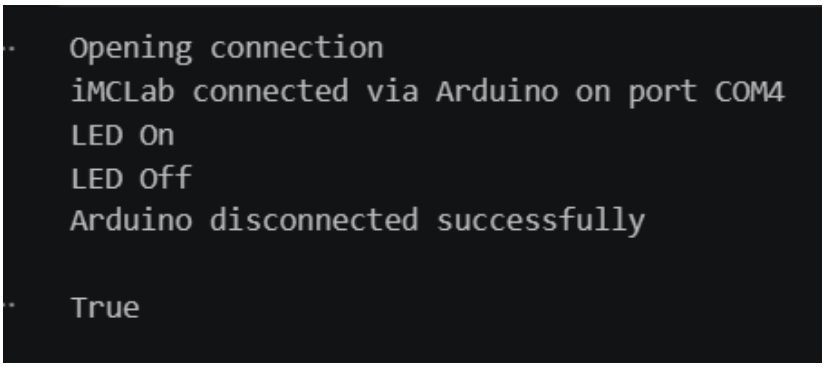
# Turn LED on
print('LED On')
a.LED(100)

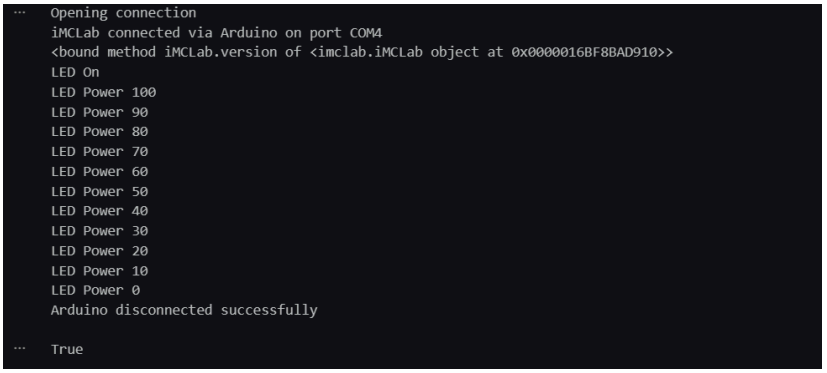
# Taper LED off
for i in range(100,-1,-10):
    print('LED Power ' + str(i))
    time.sleep(0.5)
    a.LED(i)

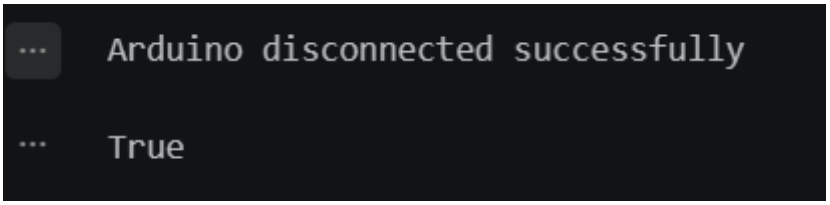
a.close()
```

```
a.close()
```

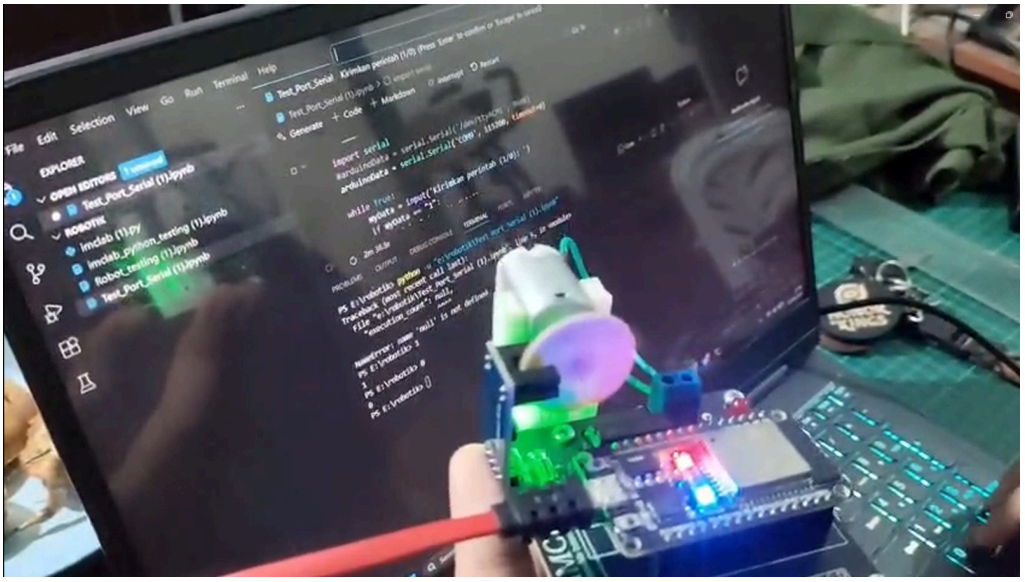
2.1.3.2. Check Serial Monitor untuk Command







2.2. Hasil

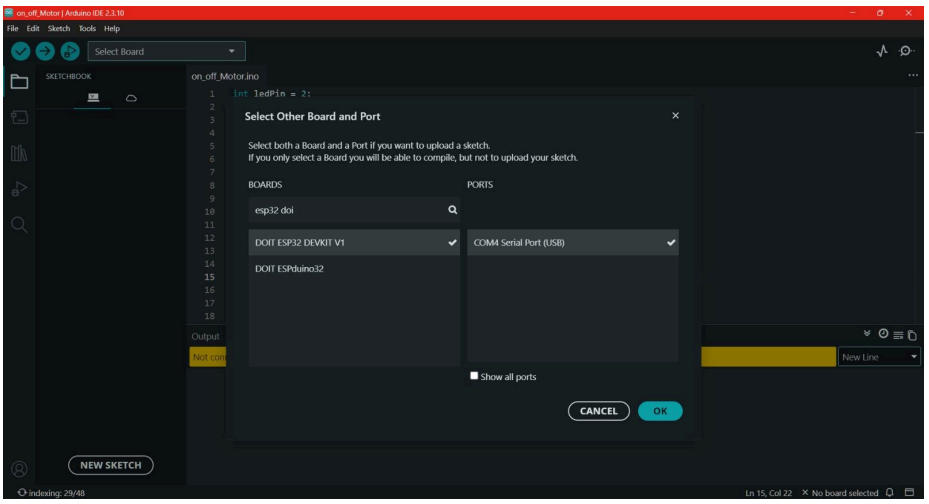


3. Blynk - Kit iMcLab

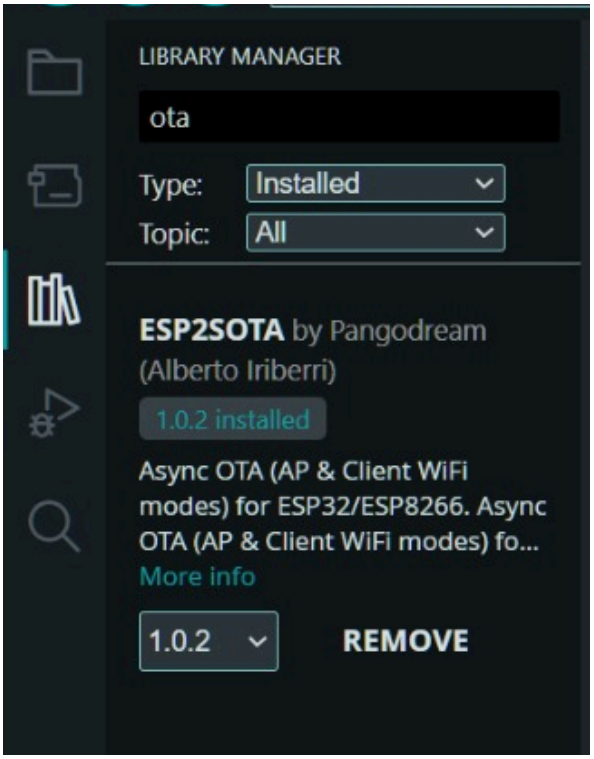
Experiment Modul Blynk menggunakan Kit iMcLab

3.1. Tata Cara

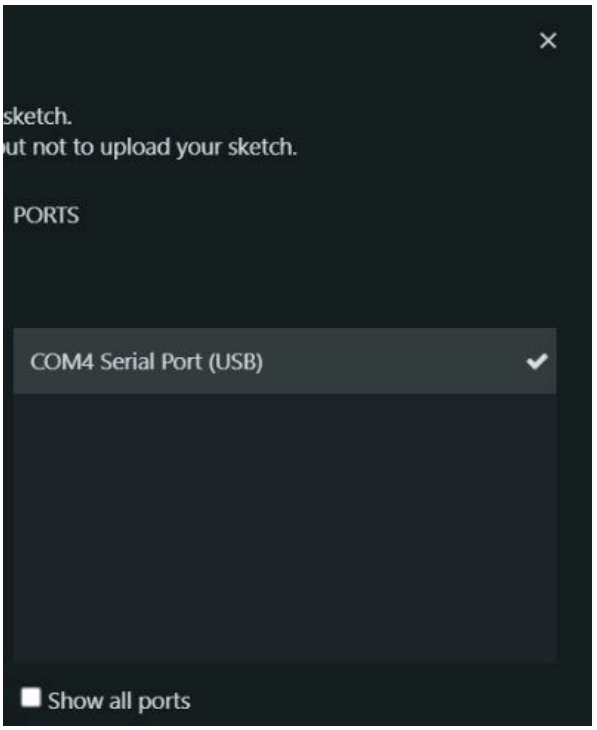
3.1.1. Setup Board Manager



3.1.2. Setup Library Manager



3.1.3. Menyambungkan Device ke Laptop



3.1.4. Start up & Compile

```
/*************************************************
*******
 * iMCLab + ESP32 + Blynk IoT
 * RPM Monitoring dan Kontrol PWM

*************************************************
******/

#define BLYNK_TEMPLATE_ID "TMPL6Hc3-3t6E"
#define BLYNK_TEMPLATE_NAME "IMCLab ROBOTIKA A081"
#define BLYNK_AUTH_TOKEN
"sgPwQPSg4unqPMTbi95NMbvQcmlctHh1"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>


//Konfigurasi Wifi
char auth[] = "sgPwQPSg4unqPMTbi95NMbvQcmlctHh1";
char ssid[] = "******";
char pass[] = "******";


//
==================================================
=========
// Motor dan LED
//
==================================================
=========

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

//
==================================================
=========
// Sensor RPM
//
==================================================
=========

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;
```

```
//
================================================
=========
// PWM
//
================================================
=========

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

//
================================================
=========
// RPM
//
================================================
=========

float rpm = 0.0;
unsigned long old_time = 0;

//
================================================
=========

String inputString = "";

BlynkTimer timer;

//
================================================
=========
// ISR Encoder
//
================================================
=========

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
```

```
  }
}

//
==================================================
=========
// Sync setelah reconnect Blynk
//
==================================================
=========

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
  Blynk.syncVirtual(V3);
}

//
==================================================
=========
// Hitung RPM
//
==================================================
=========

void hitungRPM()
{
  unsigned long cur_time = millis();

  if (cur_time - old_time >= 1000)
  {

detachInterrupt(digitalPinToInterrupt(pin_rpm));

    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

//
==================================================
=========
// Terapkan PWM ke Motor
```

```
//
=================================================
=========

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, dutyCycle);

  digitalWrite(ledPin, dutyCycle > 0);

  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

//
=================================================
=========
// Status Serial
//
=================================================
=========

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}

//
=================================================
=========
// Kirim Data ke Blynk
//
=================================================
=========

void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

//
=================================================
=========
```

```
// Slider PWM dari Blynk (V2)
//
================================================
=========

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);

  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
}

//
================================================
=========
// Tombol ON/OFF dari Blynk (V3)
//
================================================
=========

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
    if (dutyCycle == 0)
      dutyCycle = 200;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();

  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

//
================================================
=========
// Setup
//
================================================
=========
```

```
void setup()
{
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING);

  ledcSetup(
    pwmChannel,
    freq,
    resolution);

  ledcAttachPin(
    enable1Pin,
    pwmChannel);

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, 0);

  old_time = millis();

//  Blynk.begin(
//    BLYNK_AUTH_TOKEN,
//    ssid,
//    pass);

  // Koneksi ke Blynk
  Blynk.begin(auth, ssid, pass);

  timer.setInterval(
    1000L,
    kirimKeBlynk);


Serial.println("================================");
  Serial.println("iMCLab + Blynk Ready");

Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
```

```
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
}

//
==================================================
=========
// Loop
//
==================================================
=========

void loop()
{
  Blynk.run();
  timer.run();

  hitungRPM();

  while (Serial.available() > 0)
  {
    char c = Serial.read();

    if (c == '\n' || c == '\r')
    {
      inputString.trim();

      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();

        val = constrain(
          val,
          DUTY_MIN,
          DUTY_MAX);

        dutyCycle = val;
        motorEnable = (dutyCycle > 0);

        applyMotor();

        tampilkanStatus("Set PWM");
      }

      else if (inputString.length() > 0)
      {
        char cmd = inputString.charAt(0);

        if (cmd == '1')
        {
          if (dutyCycle == 0)
            dutyCycle = 200;

          motorEnable = true;
```

```
        applyMotor();

        tampilkanStatus("Motor ON");
      }

      else if (cmd == '0')
      {
        dutyCycle = 0;
        motorEnable = false;

        applyMotor();

        Serial.println("Motor OFF");
      }

      else if (cmd == '+')
      {
        dutyCycle += DUTY_STEP;

        if (dutyCycle > DUTY_MAX)
          dutyCycle = DUTY_MAX;

        motorEnable = true;

        applyMotor();

        tampilkanStatus("PWM NAIK");
      }

      else if (cmd == '-')
      {
        dutyCycle -= DUTY_STEP;

        if (dutyCycle < DUTY_MIN)
          dutyCycle = DUTY_MIN;

        motorEnable = (dutyCycle > 0);

        applyMotor();

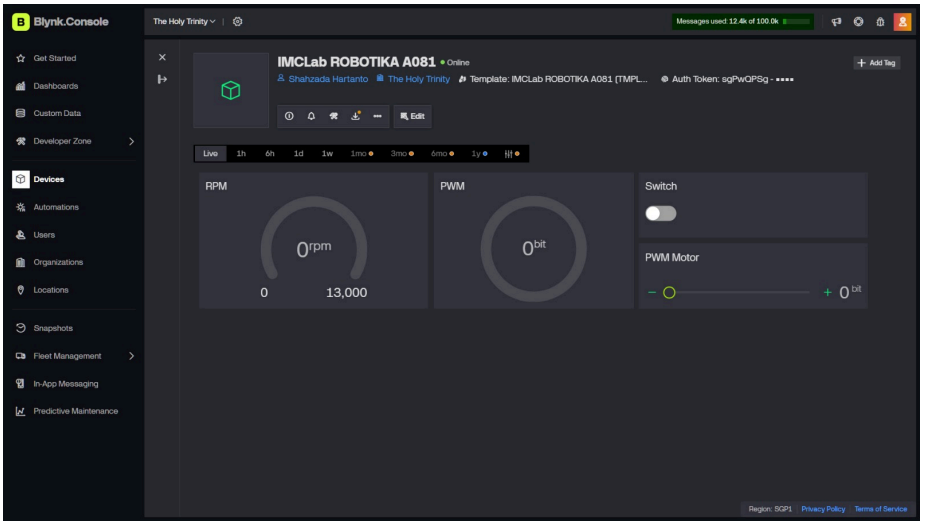
        tampilkanStatus("PWM TURUN");
      }

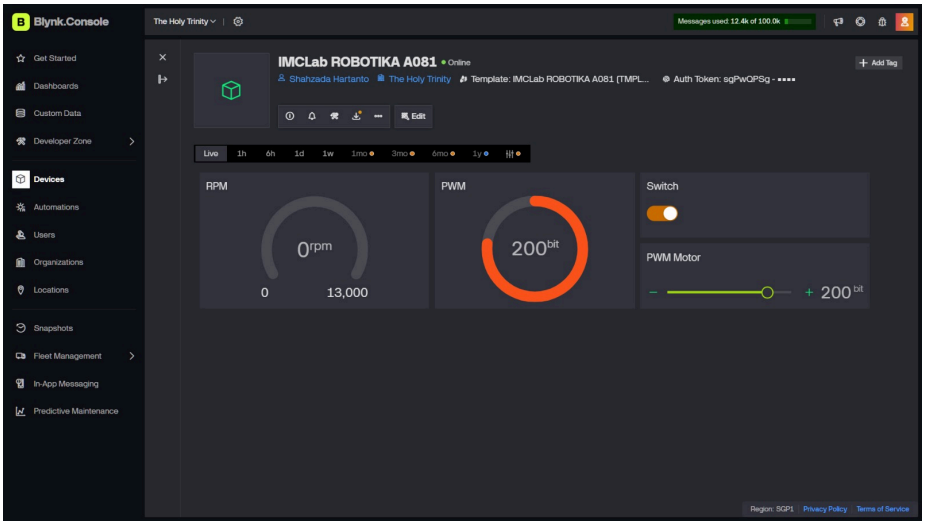
      else if (cmd == 'R' || cmd == 'r')
      {
        Serial.print("RPM = ");
        Serial.print(rpm, 1);
        Serial.print(" | PWM = ");
        Serial.println(dutyCycle);
      }
    }

    inputString = "";
  }
  else
```

```
      {
        inputString += c;
      }
    }
}
```

3.2. Dashboard Blynk Deskop





3.3. Hasil

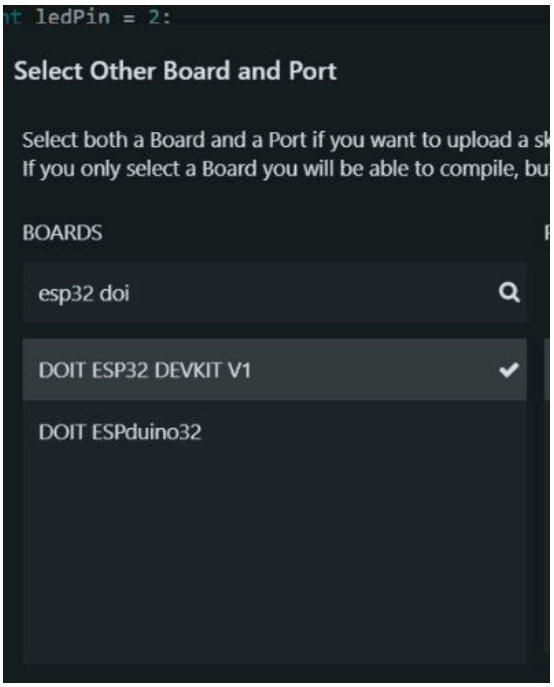


4. Blynk O.T.A - Kit iMcLab

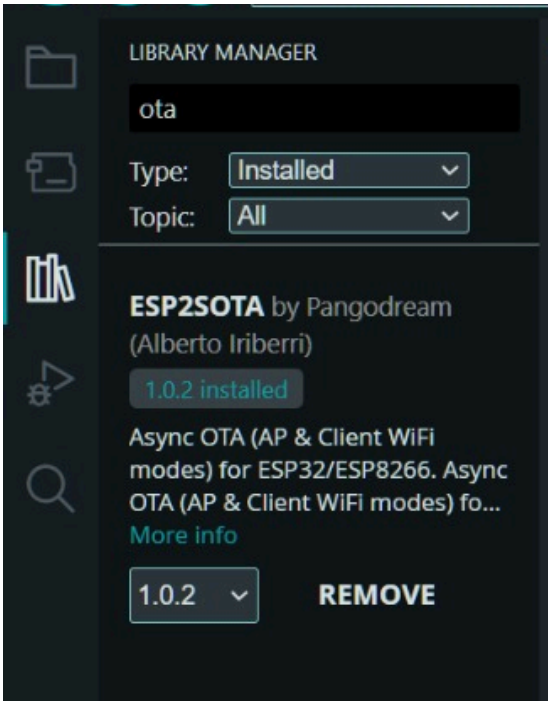
Experiment Modul O.T.A menggunakan Kit iMcLab

4.1. Tata Cara

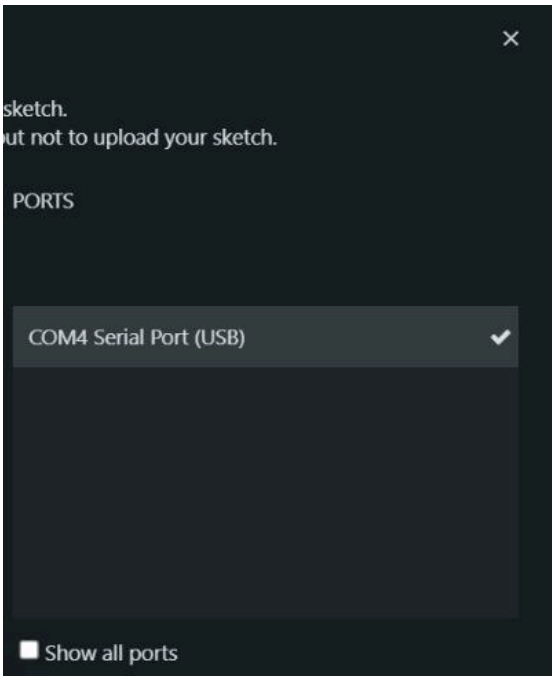
4.1.1. Setup Board Manager



4.1.2. Setup Library Manager



4.1.3. Menyambungkan Device ke Laptop



4.1.4. Code

```
// Setup Blynk
#define BLYNK_TEMPLATE_ID   "TMPL6Hc3-3t6E"
#define BLYNK_TEMPLATE_NAME "IMCLab OTA"
```

```
#define BLYNK_AUTH_TOKEN
"Apsl5rcGSuJxQSO6OVOdYre8yv8xWZdh"

// Versi firmware: WAJIB dinaikkan setiap kirim
update lewat Blynk.Air
#define BLYNK_FIRMWARE_VERSION "0.1.0"

#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <Update.h>
#include <ArduinoOTA.h>
#include <BlynkSimpleEsp32.h>

// Konfigurasi WiFi (2.4 GHz)
char auth[] = "Apsl5rcGSuJxQSO6OVOdYre8yv8xWZdh";
char ssid[] = "*****";
char pass[] = "*****";

// ArduinoOTA (jaringan lokal)
const char* OTA_HOSTNAME = "imclab-OTA";
const char* OTA_PASSWORD = "imclab123";

//
==================================================
=========
// Motor dan LED
//
==================================================
=========

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

//
==================================================
=========
// Sensor RPM
//
==================================================
=========

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;
```

```
//
================================================
=========
// PWM
//
================================================
=========

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

//
================================================
=========
// RPM
//
================================================
=========

float rpm = 0.0;
unsigned long old_time = 0;

//
================================================
=========

String inputString = "";

BlynkTimer timer;

//
================================================
=========
// OTA
//
================================================
=========

String otaUrl;
bool otaRequested = false;

//
================================================
=========
// PWM helper (kompatibel core ESP32 2.x dan 3.x)
```

```
//
=================================================
=========

void pwmBegin()
{
#if defined(ESP_ARDUINO_VERSION_MAJOR) &&
ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(enable1Pin, freq, resolution);
#else
  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(enable1Pin, pwmChannel);
#endif
}

void pwmWrite(int duty)
{
#if defined(ESP_ARDUINO_VERSION_MAJOR) &&
ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(enable1Pin, duty);
#else
  ledcWrite(pwmChannel, duty);
#endif
}

//
=================================================
=========
// ISR Encoder
//
=================================================
=========

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
}

//
=================================================
=========
// Sync setelah (re)connect Blynk
//
=================================================
=========

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
```

```
  Blynk.syncVirtual(V3);
  Blynk.virtualWrite(V4, BLYNK_FIRMWARE_VERSION);
}

//
==================================================
=========
// Hitung RPM
//
==================================================
=========

void hitungRPM()
{
  unsigned long cur_time = millis();

  if (cur_time - old_time >= 1000)
  {

detachInterrupt(digitalPinToInterrupt(pin_rpm));

    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

//
==================================================
=========
// Terapkan PWM ke Motor
//
==================================================
=========

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  pwmWrite(dutyCycle);

  digitalWrite(ledPin, dutyCycle > 0);

  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
```

```
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// Hentikan motor total (dipakai sebelum OTA)
void stopMotor()
{
  dutyCycle = 0;
  motorEnable = false;
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  pwmWrite(0);
  digitalWrite(ledPin, LOW);
}

//
==================================================
=========
// Status Serial
//
==================================================
=========

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}

//
==================================================
=========
// Kirim Data ke Blynk
//
==================================================
=========

void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

//
==================================================
=========
// Slider PWM dari Blynk (V2)
//
==================================================
=========
```

```
BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);

  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
}

//
==========================================================
=========
// Tombol ON/OFF dari Blynk (V3)
//
==========================================================
=========

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
    if (dutyCycle == 0)
      dutyCycle = 200;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();

  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

//
==========================================================
=========
// OTA 1: Blynk.Air
// Server Blynk mengirim URL file .bin ke
InternalPinOTA saat
// ada "Shipping" baru di Blynk Console. Proses
unduh dijalankan
// di loop() agar handler tidak blocking.
```

```
//
==================================================
=========

BLYNK_WRITE(InternalPinOTA)
{
  otaUrl = param.asString();
  otaRequested = true;
  Serial.println("[Blynk.Air] Permintaan OTA
diterima");
}

void runBlynkAirOTA()
{
  stopMotor();                                    //
keamanan: motor mati selama update
  detachInterrupt(digitalPinToInterrupt(pin_rpm));
  Serial.println("[Blynk.Air] URL: " + otaUrl);

  Blynk.disconnect();                             //
agar koneksi Blynk tidak mengganggu unduhan

  HTTPClient http;
  WiFiClient plainClient;
  WiFiClientSecure secureClient;

  if (otaUrl.startsWith("https"))
  {
    secureClient.setInsecure();                   //
tanpa verifikasi sertifikat (cukup untuk praktikum)
    http.begin(secureClient, otaUrl);
  }
  else
  {
    http.begin(plainClient, otaUrl);
  }

  const char* headerKeys[] = { "x-MD5" };
  http.collectHeaders(headerKeys, 1);

  int httpCode = http.GET();
  if (httpCode != HTTP_CODE_OK)
  {
    Serial.printf("[Blynk.Air] Gagal, HTTP code =
%d\n", httpCode);
    http.end();
    attachInterrupt(digitalPinToInterrupt(pin_rpm),
isr, FALLING);
    return;                                       //
lanjut jalan, Blynk akan konek ulang
  }

  int contentLength = http.getSize();
  Serial.printf("[Blynk.Air] Ukuran firmware: %d
byte\n", contentLength);
```

```
  if (contentLength <= 0 ||
!Update.begin(contentLength))
  {
    Serial.println("[Blynk.Air] Partisi OTA tidak
cukup / ukuran tidak valid");
    http.end();
    attachInterrupt(digitalPinToInterrupt(pin_rpm),
isr, FALLING);
    return;
  }

  if (http.hasHeader("x-MD5"))                    //
verifikasi integritas file
  {
    String md5 = http.header("x-MD5");
    if (md5.length() == 32)
    {
      md5.toLowerCase();
      Update.setMD5(md5.c_str());
      Serial.println("[Blynk.Air] MD5: " + md5);
    }
  }

  unsigned long t0 = millis();
  size_t written =
Update.writeStream(http.getStream());
  http.end();

  Serial.printf("[Blynk.Air] Tertulis %u / %d byte
dalam %lu ms\n",
                (unsigned)written, contentLength,
millis() - t0);

  if (written != (size_t)contentLength ||
!Update.end() || !Update.isFinished())
  {
    Serial.printf("[Blynk.Air] Update gagal, error
#%d\n", Update.getError());
    attachInterrupt(digitalPinToInterrupt(pin_rpm),
isr, FALLING);
    return;
  }

  Serial.println("[Blynk.Air] Update sukses,
reboot...");
  delay(500);
  ESP.restart();
}

//
================================================
=========
// OTA 2: ArduinoOTA (WiFi lokal, port jaringan di
Arduino IDE)
```

```
//
=================================================
=========

void setupArduinoOTA()
{
  ArduinoOTA.setHostname(OTA_HOSTNAME);
  ArduinoOTA.setPassword(OTA_PASSWORD);

  ArduinoOTA.onStart([]() {
    stopMotor();

detachInterrupt(digitalPinToInterrupt(pin_rpm));
    Serial.println("[ArduinoOTA] Mulai update");
  });
  ArduinoOTA.onProgress([](unsigned int progress,
unsigned int total) {
    Serial.printf("[ArduinoOTA] Progres: %u%%\r",
(progress * 100) / total);
  });
  ArduinoOTA.onEnd([]() {
    Serial.println("\n[ArduinoOTA] Selesai,
reboot...");
  });
  ArduinoOTA.onError([](ota_error_t error) {
    Serial.printf("[ArduinoOTA] Error[%u]\n",
error);
  });

  ArduinoOTA.begin();
}

//
=================================================
=========
// Setup
//
=================================================
=========

void setup()
{
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
```

```
    FALLING);

  pwmBegin();

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  pwmWrite(0);

  old_time = millis();

  // Koneksi ke Blynk
  Blynk.begin(auth, ssid, pass);

  Serial.print("IP: ");
  Serial.println(WiFi.localIP());

  setupArduinoOTA();

  timer.setInterval(
    1000L,
    kirimKeBlynk);


Serial.println("================================");
  Serial.println("iMCLab + Blynk + OTA Ready");
  Serial.println("Firmware v"
BLYNK_FIRMWARE_VERSION);

Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
  Serial.println("V = lihat versi firmware");
}

//
==================================================
=========
// Loop
//
==================================================
=========

void loop()
{
  if (otaRequested)
  {
    otaRequested = false;
    runBlynkAirOTA();
  }
```

```
Blynk.run();
ArduinoOTA.handle();
timer.run();

hitungRPM();

while (Serial.available() > 0)
{
  char c = Serial.read();

  if (c == '\n' || c == '\r')
  {
    inputString.trim();

    if (inputString.startsWith("PWM:"))
    {
      int val = inputString.substring(4).toInt();

      val = constrain(
        val,
        DUTY_MIN,
        DUTY_MAX);

      dutyCycle = val;
      motorEnable = (dutyCycle > 0);

      applyMotor();

      tampilkanStatus("Set PWM");
    }

    else if (inputString.length() > 0)
    {
      char cmd = inputString.charAt(0);

      if (cmd == '1')
      {
        if (dutyCycle == 0)
          dutyCycle = 200;

        motorEnable = true;

        applyMotor();

        tampilkanStatus("Motor ON");
      }

      else if (cmd == '0')
      {
        dutyCycle = 0;
        motorEnable = false;

        applyMotor();

        Serial.println("Motor OFF");
```

```
        }

        else if (cmd == '+')
        {
          dutyCycle += DUTY_STEP;

          if (dutyCycle > DUTY_MAX)
            dutyCycle = DUTY_MAX;

          motorEnable = true;

          applyMotor();

          tampilkanStatus("PWM NAIK");
        }

        else if (cmd == '-')
        {
          dutyCycle -= DUTY_STEP;

          if (dutyCycle < DUTY_MIN)
            dutyCycle = DUTY_MIN;

          motorEnable = (dutyCycle > 0);

          applyMotor();

          tampilkanStatus("PWM TURUN");
        }

        else if (cmd == 'R' || cmd == 'r')
        {
          Serial.print("RPM = ");
          Serial.print(rpm, 1);
          Serial.print(" | PWM = ");
          Serial.println(dutyCycle);
        }

        else if (cmd == 'V' || cmd == 'v')
        {
          Serial.println("Firmware v"
BLYNK_FIRMWARE_VERSION);
        }
      }

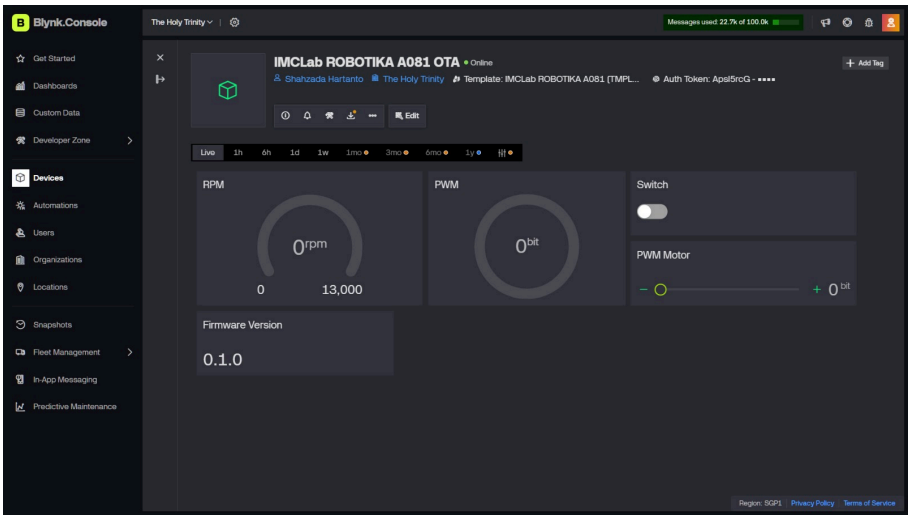
      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

_Blynk_iMCLab.ino

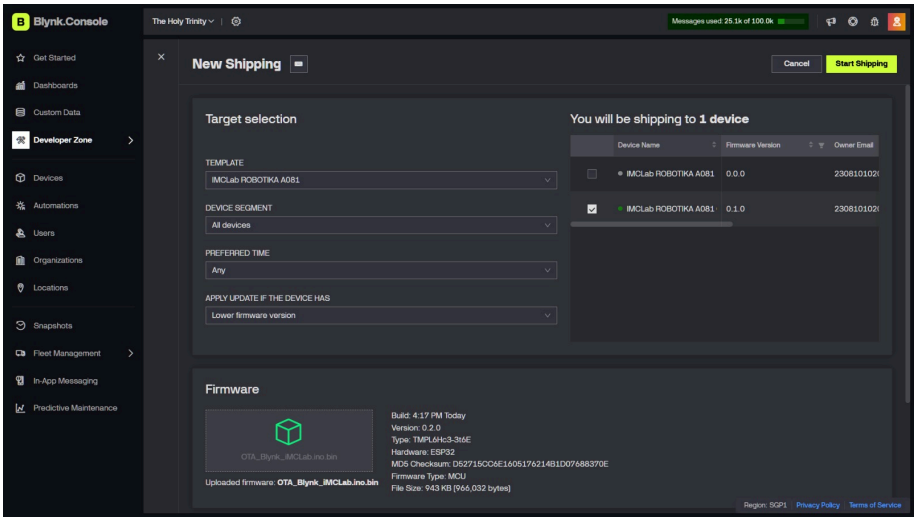
```
// Setup Blynk
#define BLYNK_TEMPLATE_ID   "TMPL6Hc3-3t6E"
#define BLYNK_TEMPLATE_NAME "IMCLab OTA"
#define BLYNK_AUTH_TOKEN    "Apsl5rcGSuJxQSO6OVOdYre8yv8xWZdh"

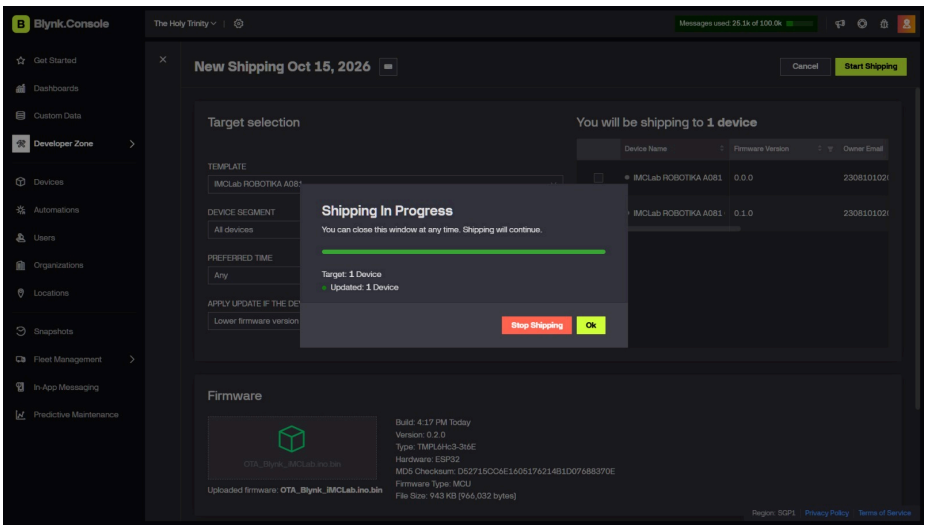
// Versi firmware: WAJIB dinaikkan setiap kirim update lewat Blynk.Air
#define BLYNK_FIRMWARE_VERSION "0.2.0"
```

4.2. Dashboard Blynk OTA



4.3. Update via Blynk Air OTA





4.4. Hasil setelah Update via Blynk Air OTA

